Analiza Architektury - Amazonix

1. Opis projektu:

Amazonix to nowoczesna platforma e-commerce (sklep internetowy) zbudowana w architekturze MVC (Model-View-Controller). Aplikacja umożliwia przeglądanie produktów, zarządzanie koszykiem zakupowym oraz autoryzację użytkowników.

2. ARCHITEKTURA I TECHNOLOGIE

- Język programowania: PHP 8.4
- Baza danych: MySQL / MariaDB (import pliku amazonix.sql)
- Architektura: MVC (src/Controllers, src/View, templates)
- Routing: Własny system routingu (config/routes.php) z obsługą parametrów i grup tras.
- Frontend: HTML, CSS, JavaScript (Bootstrap, autorskie style w public/static).
- Autoryzacja: System oparty na sesjach oraz tokenach JWT.
- Integracja API: Projekt przewiduje integrację z zewnętrznymi serwisami (np. moduł python_api).

3. GŁÓWNE FUNKCJONALNOŚCI

- Katalog produktów: Wyświetlanie listy produktów, filtrowanie według kategorii oraz szczegółowy podgląd pojedynczego produktu.
- System użytkowników: Rejestracja nowych kont, logowanie, wylogowanie oraz zarządzanie profilem (AccountController).
- Koszyk zakupowy: Dodawanie produktów do koszyka, zmiana ilości (zwiększanie/zmniejszanie) oraz usuwanie pozycji.
- Obsługa sesji: Bezpieczne przechowywanie stanu logowania i komunikatów (toast messages).

4. STRUKTURA KATALOGÓW

- /src: Logika biznesowa, kontrolery, serwisy i helpery.
- /templates: Pliki widoków i szablony HTML.
- /public: Pliki dostępne publicznie (.htaccess, index.php, assety).
- /config: Definicje tras i konfiguracja specyficzna dla aplikacji.
- /bootstrap: Skrypty startowe aplikacji (inicjalizacja usług).

5. Baza danych - struktura

- **Tabele:**
  - **customers**

#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra
1	**id**	int(11)			No	*None*		AUTO_INCREMENT
2	**first_name**	varchar(50)	utf8mb4_general_ci		Yes	*NULL*		
3	**last_name**	varchar(50)	utf8mb4_general_ci		Yes	*NULL*		
4	**phone**	varchar(30)	utf8mb4_general_ci		Yes	*NULL*		
5	**country**	varchar(50)	utf8mb4_general_ci		Yes	*NULL*		
6	**city**	varchar(50)	utf8mb4_general_ci		Yes	*NULL*		
7	**address**	varchar(150)	utf8mb4_general_ci		Yes	*NULL*		
8	**user_id**	int(11)			Yes	*NULL*		

  - **cart**

#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra
1	**id**	int(11)			No	*None*		AUTO_INCREMENT
2	**user_id**	int(11)			No	*None*		
3	**product_id**	int(11)			No	*None*		
4	**quantity**	int(11)			No	1		
5	**created_at**	datetime			No	current_timestamp()		

- **deliveries**

#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra
1	id	int(11)			No	None		AUTO_INCREMENT
2	order_id	int(11)			No	None		
3	warehouse_id	int(11)			No	None		
4	vehicle_id	int(11)			Yes	NULL		
5	driver_id	int(11)			Yes	NULL		
6	status	enum('planned', 'in_transit', 'delivered')	utf8mb4_general_ci		Yes	planned		
7	departure_date	datetime			Yes	NULL		
8	delivery_date	datetime			Yes	NULL		

- **delivery_items**

#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra
1	id	int(11)			No	None		AUTO_INCREMENT
2	delivery_id	int(11)			No	None		
3	product_id	int(11)			No	None		
4	quantity	int(11)			No	None		

- **employees**

#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra
1	id	int(11)			No	None		AUTO_INCREMENT
2	first_name	varchar(50)	utf8mb4_general_ci		Yes	NULL		
3	last_name	varchar(50)	utf8mb4_general_ci		Yes	NULL		
4	warehouse_id	int(11)			Yes	NULL		
5	user_id	int(11)			Yes	NULL		

- **orders**

#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra
1	id	int(11)			No	None		AUTO_INCREMENT
2	customer_id	int(11)			No	None		
3	order_date	datetime			Yes	current_timestamp()		
4	status	enum('new', 'paid', 'shipped', 'delivered', 'cance...	utf8mb4_general_ci		Yes	new		
5	total_price	decimal(10,2)			Yes	NULL		

- **order_items**

#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra
1	id	int(11)			No	None		AUTO_INCREMENT
2	order_id	int(11)			No	None		
3	product_id	int(11)			No	None		
4	quantity	int(11)			No	None		
5	price_at_time	decimal(10,2)			No	None		

- **products**

#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra
1	id	int(11)			No	None		AUTO_INCREMENT
2	name	varchar(150)	utf8mb4_general_ci		No	None		
3	sku	varchar(50)	utf8mb4_general_ci		No	None		
4	description	text	utf8mb4_general_ci		Yes	NULL		
5	price	decimal(10,2)			No	None		
6	weight	decimal(10,2)			Yes	NULL		
7	category_id	int(11)			Yes	NULL		
8	active	tinyint(1)			Yes	1		
9	image	text	utf8mb4_general_ci		Yes	NULL		
10	rating	tinyint(1)			Yes	NULL		

- **product_categories**

#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra
1	id	int(11)			No	None		AUTO_INCREMENT
2	name	varchar(100)	utf8mb4_general_ci		No	None		

- **roles**

#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra
1	id	int(11)			No	None		AUTO_INCREMENT
2	name	varchar(50)	utf8mb4_general_ci		No	None		
3	salary	float			No	4806		
4	action	varchar(255)	utf8mb4_general_ci		No	None		

- **stock_movement**

#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra
1	id	int(11)			No	None		AUTO_INCREMENT
2	product_id	int(11)			No	None		
3	warehouse_id	int(11)			No	None		
4	movement_type	enum('in', 'out', 'transfer')	utf8mb4_general_ci		No	None		
5	quantity	int(11)			No	None		
6	reference_table	varchar(50)	utf8mb4_general_ci		Yes	NULL		
7	reference_id	int(11)			Yes	NULL		
8	created_at	datetime			Yes	current_timestamp()		

- **users**

#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra
1	id	int(11)			No	None		AUTO_INCREMENT
2	email	varchar(255)	utf8mb4_general_ci		No	None		
3	password	varchar(255)	utf8mb4_general_ci		No	None		
4	role_id	int(11)			No	None		
5	created_at	datetime			No	current_timestamp()		

- **vehicles**

#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra
1	id	int(11)			No	None		AUTO_INCREMENT
2	type	enum('truck', 'van')	utf8mb4_general_ci		No	None		
3	registration_number	varchar(30)	utf8mb4_general_ci		No	None		
4	capacity	int(11)			No	None		

- **warehouses**

#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra
1	id	int(11)			No	None		AUTO_INCREMENT
2	name	varchar(100)	utf8mb4_general_ci		No	None		
3	country	varchar(50)	utf8mb4_general_ci		No	None		
4	city	varchar(50)	utf8mb4_general_ci		No	None		
5	address	varchar(150)	utf8mb4_general_ci		Yes	NULL		
6	capacity	int(11)			Yes	NULL		

- **warehouses_products**

#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra
1	id	int(11)			No	None		AUTO_INCREMENT
2	warehouse_id	int(11)			No	None		
3	product_id	int(11)			No	None		
4	quantity	int(11)			No	0		
5	reserved_quantity	int(11)			No	0		

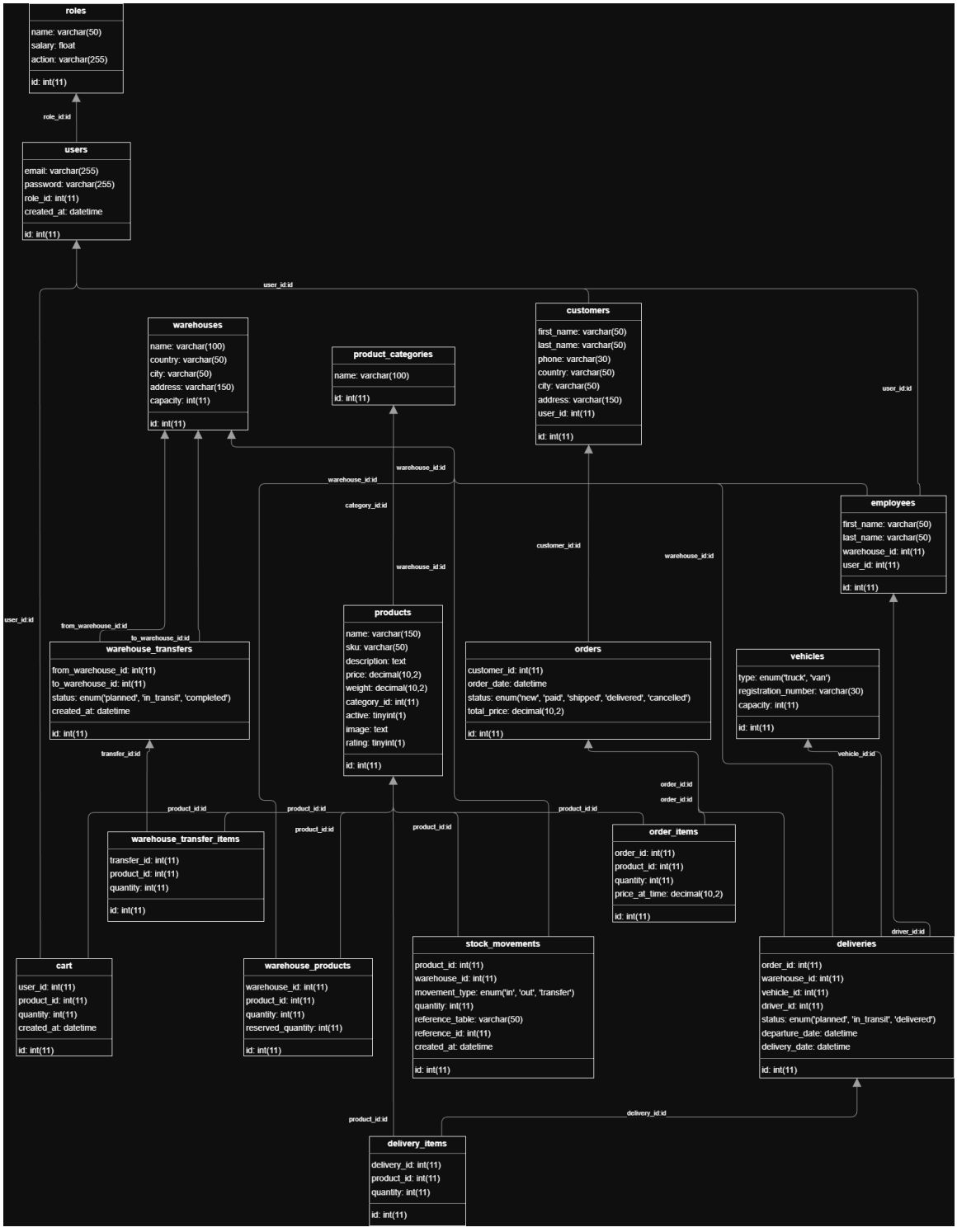
- **warehouses_transfers**

#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra
1	id	int(11)			No	None		AUTO_INCREMENT
2	from_warehouse_id	int(11)			No	None		
3	to_warehouse_id	int(11)			No	None		
4	status	enum('planned', 'in_transit', 'completed')	utf8mb4_general_ci		Yes	planned		
5	created_at	datetime			Yes	current_timestamp()		

- **warehouse_transfer_items**

#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra
1	id	int(11)			No	None		AUTO_INCREMENT
2	transfer_id	int(11)			No	None		
3	product_id	int(11)			No	None		
4	quantity	int(11)			No	None		

- **Diagram:**


roles
name: varchar(50)
salary: float
action: varchar(255)
id: int(11)
role_id:id
users
email: varchar(255)
password: varchar(255)
role_id: int(11)
created_at: datetime
id: int(11)
user_id:id
warehouses
name: varchar(100)
country: varchar(50)
city: varchar(50)
address: varchar(150)
capacity: int(11)
id: int(11)
product_categories
name: varchar(100)
id: int(11)
customers
first_name: varchar(50)
last_name: varchar(50)
phone: varchar(30)
country: varchar(50)
city: varchar(50)
address: varchar(150)
user_id: int(11)
id: int(11)
warehouse_id:id
category_id:id
customer_id:id
employees
first_name: varchar(50)
last_name: varchar(50)
warehouse_id: int(11)
user_id: int(11)
id: int(11)
from_warehouse_id:id
to_warehouse_id:id
warehouse_transfers
from_warehouse_id: int(11)
to_warehouse_id: int(11)
status: enum('planned', 'in_transit', 'completed')
created_at: datetime
id: int(11)
products
name: varchar(150)
sku: varchar(50)
description: text
price: decimal(10,2)
weight: decimal(10,2)
category_id: int(11)
active: tinyint(1)
image: text
rating: tinyint(1)
id: int(11)
orders
customer_id: int(11)
order_date: datetime
status: enum('new', 'paid', 'shipped', 'delivered', 'cancelled')
total_price: decimal(10,2)
id: int(11)
vehicles
type: enum('truck', 'van')
registration_number: varchar(30)
capacity: int(11)
id: int(11)
transfer_id:id
product_id:id
order_id:id
vehicle_id:id
warehouse_transfer_items
transfer_id: int(11)
product_id: int(11)
quantity: int(11)
id: int(11)
order_items
order_id: int(11)
product_id: int(11)
quantity: int(11)
price_at_time: decimal(10,2)
id: int(11)
driver_id:id
cart
user_id: int(11)
product_id: int(11)
quantity: int(11)
created_at: datetime
id: int(11)
warehouse_products
warehouse_id: int(11)
product_id: int(11)
quantity: int(11)
reserved_quantity: int(11)
id: int(11)
stock_movements
product_id: int(11)
warehouse_id: int(11)
movement_type: enum('in', 'out', 'transfer')
quantity: int(11)
reference_table: varchar(50)
reference_id: int(11)
created_at: datetime
id: int(11)
deliveries
order_id: int(11)
warehouse_id: int(11)
vehicle_id: int(11)
driver_id: int(11)
status: enum('planned', 'in_transit', 'delivered')
departure_date: datetime
delivery_date: datetime
id: int(11)
delivery_id:id
delivery_items
delivery_id: int(11)
product_id: int(11)
quantity: int(11)
id: int(11)

6. Gałęzie- opis, najważniejsze pliki

a. Gałąź: main

Oficjalna, stabilna wersja projektu. Zawiera kod gotowy do wdrożenia, będący wynikiem cykli rozwojowych przeprowadzonych na branchach feature i develop.

b. Gałąź: develop

Główna gałąź integracyjna projektu. Służy do gromadzenia wszystkich ukończonych i przetestowanych funkcjonalności przed ich połączeniem z gałęzią główną (main). To tutaj następuje finalne łączenie logiki biznesowej, API i interfejsu użytkownika.

c. Gałąź: feature/Python-db-connection

Branch Python-db-connection został stworzony w celu izolacji prac nad konkretną funkcjonalnością lub poprawką. Jego głównym zadaniem jest rozwój i testowanie zmian w wydzielonym środowisku, co gwarantuje stabilność głównego kodu aplikacji do momentu pełnej weryfikacji nowości.

Najważniejsze pliki:

python_conn/app.py

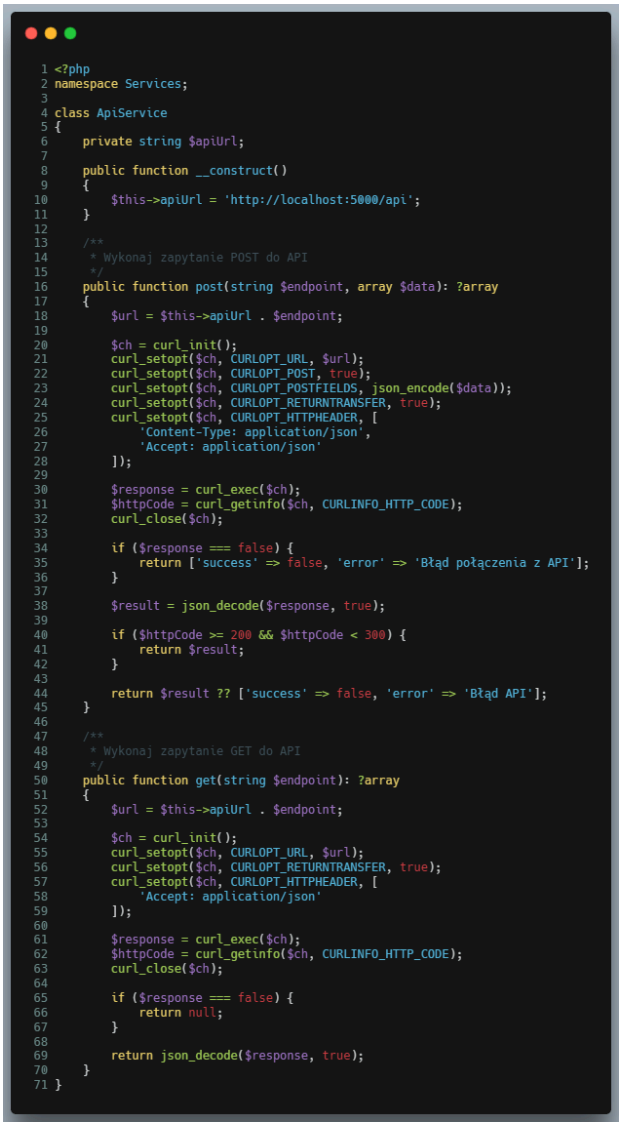
```python
import mysql.connector


def connect_to_database():
    try:
        conn = mysql.connector.connect(
            host="localhost",
            user="root",
            password="",
            database="warehouse_system"
        )

        if conn.is_connected():
            print("Połączenie z bazą działa")

        return conn

    except mysql.connector.Error as err:
        print("Błąd połączenia:", err)
        return None


conn = connect_to_database()

if conn:
    conn.close()
    print("Połączenie zamknięte")
```

d. Gałąź: feature/api-python

Branch ten skupiał się na integracji backendu PHP z API napisanym w Pythonie. Wprowadzono usługę ApiService, która obsługuje komunikację z zewnętrznym modułem danych, pobieranie listy produktów oraz szyfrowanie i deszyfrowanie tokenów JWT używanych do autoryzacji.

Najważniejsze pliki:

src/Services/ApiService.php



```php
<?php
namespace Services;

class ApiService
{
    private string $apiUrl;

    public function __construct()
    {
        $this->apiUrl = 'http://localhost:5000/api';
    }

    /**
     * Wykonaj zapytanie POST do API
     */
    public function post(string $endpoint, array $data): ?array
    {
        $url = $this->apiUrl . $endpoint;

        $ch = curl_init();
        curl_setopt($ch, CURLOPT_URL, $url);
        curl_setopt($ch, CURLOPT_POST, true);
        curl_setopt($ch, CURLOPT_POSTFIELDS, json_encode($data));
        curl_setopt($ch, CURLOPT_RETURNTRANSFER, true);
        curl_setopt($ch, CURLOPT_HTTPHEADER, [
            'Content-Type: application/json',
            'Accept: application/json'
        ]);

        $response = curl_exec($ch);
        $httpCode = curl_getinfo($ch, CURLINFO_HTTP_CODE);
        curl_close($ch);

        if ($response === false) {
            return ['success' => false, 'error' => 'Błąd połączenia z API'];
        }

        $result = json_decode($response, true);

        if ($httpCode >= 200 && $httpCode < 300) {
            return $result;
        }

        return $result ?? ['success' => false, 'error' => 'Błąd API'];
    }

    /**
     * Wykonaj zapytanie GET do API
     */
    public function get(string $endpoint): ?array
    {
        $url = $this->apiUrl . $endpoint;

        $ch = curl_init();
        curl_setopt($ch, CURLOPT_URL, $url);
        curl_setopt($ch, CURLOPT_RETURNTRANSFER, true);
        curl_setopt($ch, CURLOPT_HTTPHEADER, [
            'Accept: application/json'
        ]);

        $response = curl_exec($ch);
        $httpCode = curl_getinfo($ch, CURLINFO_HTTP_CODE);
        curl_close($ch);

        if ($response === false) {
            return null;
        }

        return json_decode($response, true);
    }
}
```

src/Controllers/AuthController.php

```php
<?php
declare(strict_types=1);
namespace App\Controllers;

use App\View\View;
use Services\ApiService;

class AuthController
{
    private View $view;
    private ApiService $api;

    public function __construct(View $view)
    {
        $this->view = $view;
        $this->api = new ApiService();

        if (session_status() === PHP_SESSION_NONE) {
            session_start();
        }
    }

    public function showLoginForm(): void
    {
        $this->view->renderPage('login', [
            'title' => 'Logowanie - Amazonix',
            'scripts' => ['static/js/login.js'],
            'styles' => []
        ]);
    }

    public function showRegisterForm(): void
    {
        $this->view->renderPage('register', [
            'title' => 'Rejestracja - Amazonix',
            'scripts' => ['static/js/register.js'],
            'styles' => []
        ]);
    }

    public function login(): void
    {
        $email = $_POST['email'] ?? '';
        $password = $_POST['password'] ?? '';

        $result = $this->api->post('/login', [
            'email' => $email,
            'password' => $password
        ]);

        if ($result && isset($result['success']) && $result['success'] === true) {
            $_SESSION['user_id'] = $result['user_id'];
            $_SESSION['user_email'] = $result['email'];
            $_SESSION['user_name'] = ($result['first_name'] ?? '') . ' ' . ($result['last_name'] ?? '');
            $_SESSION['user_role'] = $result['role_name'] ?? '';

            $_SESSION['login_success'] = "Witaj, " . $_SESSION['user_name'] . "!";
            header('Location: /amazonix/');
            exit;
        } else {
            $_SESSION['login_error'] = $result['error'] ?? 'Nieprawidłowe dane logowania';
            header('Location: /amazonix/login');
            exit;
        }
    }

    public function register(): void
    {
        $userData = [
            'email' => $_POST['email'] ?? '',
            'password' => $_POST['password'] ?? '',
            'first_name' => $_POST['first_name'] ?? '',
            'last_name' => $_POST['last_name'] ?? '',
            'phone' => $_POST['phone'] ?? '',
            'country' => $_POST['country'] ?? '',
            'city' => $_POST['city'] ?? '',
            'address' => $_POST['address'] ?? ''
        ];

        // Walidacja haseł
        if ($userData['password'] !== ($_POST['repassword'] ?? '')) {
            $_SESSION['register_error'] = 'Hasła nie są identyczne!';
            header('Location: /amazonix/register');
            exit;
        }

        $result = $this->api->post('/register', $userData);

        if ($result && isset($result['success']) && $result['success'] === true) {

            $loginResult = $this->api->post('/login', [
                'email' => $userData['email'],
                'password' => $userData['password']
            ]);

            if ($loginResult && isset($loginResult['success']) && $loginResult['success'] === true) {
                $_SESSION['user_id'] = $loginResult['user_id'];
                $_SESSION['user_email'] = $loginResult['email'];
                $_SESSION['user_name'] = ($loginResult['first_name'] ?? '') . ' ' . ($loginResult['last_name'] ?? '');
                $_SESSION['user_role'] = $loginResult['role_name'] ?? '';

                $_SESSION['login_success'] = "Witaj, " . $_SESSION['user_name'] . "!";
                header('Location: /amazonix/');
                exit;
            } else {
                $_SESSION['login_success'] = 'Rejestracja udana! Możesz się zalogować.';
                header('Location: /amazonix/login');
                exit;
            }
        } else {
            $_SESSION['register_error'] = $result['error'] ?? 'Błąd rejestracji';
            header('Location: /amazonix/register');
            exit;
        }
    }

    public function logout(): void
    {
        session_destroy();
        header('Location: /amazonix/');
        exit;
    }
}
```

python_api/app.py

```python
from flask import Flask, jsonify
from flask_cors import CORS
from endpoints import products, auth

app = Flask(__name__)
CORS(app)  # Pozwala na zapytania z frontendu

@app.route('/')
def home():
    return jsonify({
        'message': 'API Amazonix działa!',
        'endpoints': {
            'produkty': '/api/produkty [GET]',
            'login': '/api/login [POST]',
            'register': '/api/register [POST]'
        }
    })

@app.route('/api/produkty', methods=['GET'])
def api_get_products():
    return products.get_products()

@app.route('/api/login', methods=['POST'])
def api_login():
    return auth.login()

@app.route('/api/register', methods=['POST'])
def api_register():
    return auth.register()

if __name__ == '__main__':
    print("\n" + "="*50)
    print("🚀 API AMAZONIX")
    print("="*50)
    print("📡 Serwer działa na: http://localhost:5000")
    print("📦 Endpoint produktów: http://localhost:5000/api/produkty")
    print("="*50 + "\n")
    app.run(debug=True, port=5000, host='0.0.0.0')
```

e. Gałąź: feature/strona-glowna

Gałąź odpowiedzialna za gruntowną przebudowę strony głównej. Wprowadzono dynamiczne generowanie kart produktów, nowy layout nawigacji z animacjami. Prace objęły również optymalizację renderingu i zmianę palety kolorystycznej serwisu.

Najważniejsze pliki:

src/Controllers/MainController.php

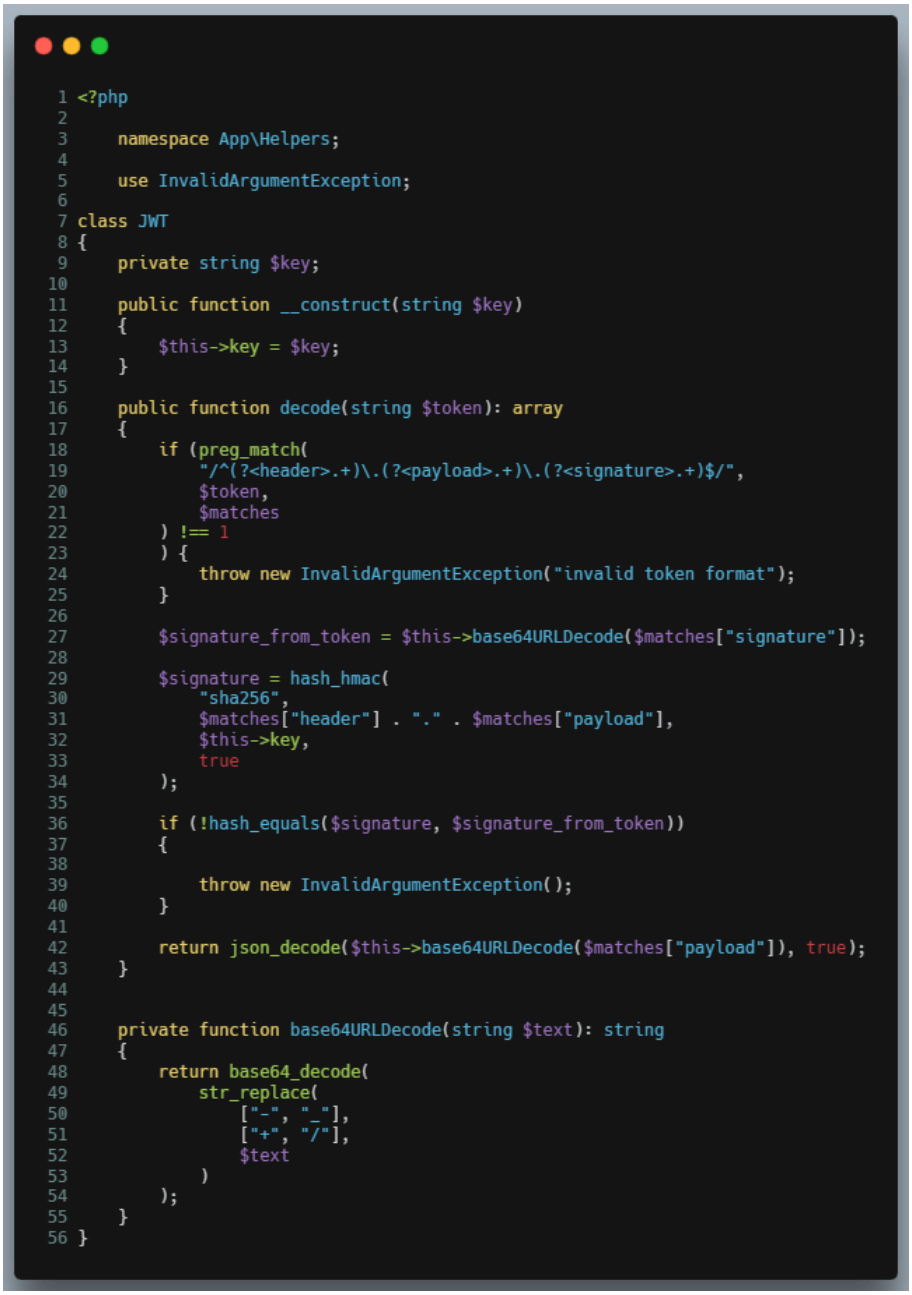
```php
<?php
    declare(strict_types=1);

    namespace App\Controllers;
    use App\View\View;


    class SiteController
    {
        private View $view;

        public function __construct(View $view)
        {
            $this->view = $view;
        }

        public function home(): void
        {
            $products = [
                [
                    'name' => 'Bezprzewodowe słuchawki Bluetooth',
                    'price' => 49.99,
                    'image' => 'public/static/img/products/wireless-bluetooth-headphones.webp'
                ],
                [
                    'name' => 'Smart Fitness Watch Pro',
                    'price' => 129.99,
                    'image' => 'public/static/img/products/smart-fitness-watch-pro.webp'
                ],
                [
                    'name' => 'Przenośny głośnik Bluetooth',
                    'price' => 29.99,
                    'image' => 'public/static/img/products/portable-speaker-with-bluetooth.webp'
                ],
                [
                    'name' => 'Klawiatura z oświetleniem RGB',
                    'price' => 19.99,
                    'image' => 'public/static/img/products/keyboard-with-rgb-light.webp'
                ]

            ];
            $this->view->renderPage('home', [
                'title' => 'Amazonix',
                'products' => $products,
                'scripts' => [
                    'static/js/search-bar.js'
                ],
                'styles' => [
                    'public/static/css/style.css'
                ]
            ]);
        }

    }
```

f. Gałąź: feature/login-z-jwt

Branch ten zrealizował przejście na bezstanowe uwierzytelnianie użytkowników przy użyciu tokenów JWT. Kluczowe zmiany objęły usunięcie tradycyjnych sesji PHP na rzecz bezpiecznej wymiany tokenów z API, co umożliwia łatwiejsze skalowanie aplikacji. Dodatkowo zintegrowano widoki logowania i rejestracji z nową logiką autoryzacji.

Najważniejsze pliki:

src/Helpers/JWT.php



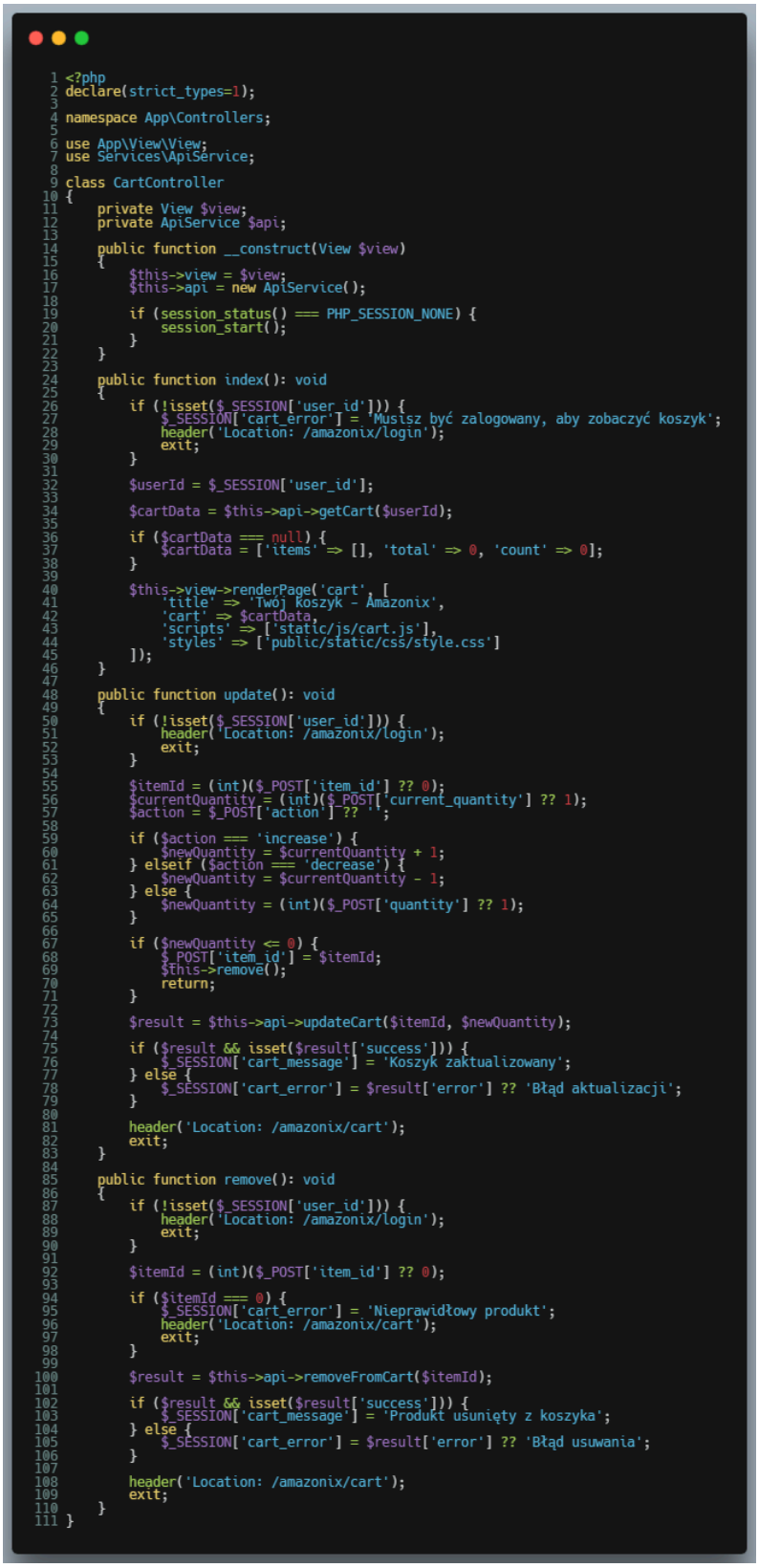
```php
<?php

    namespace App\Helpers;

    use InvalidArgumentException;

class JWT
{
    private string $key;

    public function __construct(string $key)
    {
        $this->key = $key;
    }

    public function decode(string $token): array
    {
        if (preg_match(
            "/^(?<header>.+)\.(?<payload>.+)\.(?<signature>.+)$/",
            $token,
            $matches
        ) !== 1
        ) {
            throw new InvalidArgumentException("invalid token format");
        }

        $signature_from_token = $this->base64URLDecode($matches["signature"]);

        $signature = hash_hmac(
            "sha256",
            $matches["header"] . "." . $matches["payload"],
            $this->key,
            true
        );

        if (!hash_equals($signature, $signature_from_token))
        {

            throw new InvalidArgumentException();
        }

        return json_decode($this->base64URLDecode($matches["payload"]), true);
    }


    private function base64URLDecode(string $text): string
    {
        return base64_decode(
            str_replace(
                ["-", "_"],
                ["+", "/"],
                $text
            )
        );
    }
}
```

g. Gałąź: feature/koszyk-zakupowy

Głównym celem było wdrożenie pełnej obsługi koszyka zakupowego. Prace objęły logikę dodawania produktów, dynamiczną zmianę ilości oraz usuwanie pozycji z zamówienia. Branch ten połączył backendową logikę przeliczania cen z frontendowym interfejsem użytkownika, umożliwiając płynne zarządzanie zakupami przed finalizacją.

Najważniejsze pliki:

src/Controllers/CartController.php



```php
<?php
declare(strict_types=1);

namespace App\Controllers;

use App\View\View;
use Services\ApiService;

class CartController
{
    private View $view;
    private ApiService $api;

    public function __construct(View $view)
    {
        $this->view = $view;
        $this->api = new ApiService();

        if (session_status() === PHP_SESSION_NONE) {
            session_start();
        }
    }

    public function index(): void
    {
        if (!isset($_SESSION['user_id'])) {
            $_SESSION['cart_error'] = 'Musisz być zalogowany, aby zobaczyć koszyk';
            header('Location: /amazonix/login');
            exit;
        }

        $userId = $_SESSION['user_id'];

        $cartData = $this->api->getCart($userId);

        if ($cartData === null) {
            $cartData = ['items' => [], 'total' => 0, 'count' => 0];
        }

        $this->view->renderPage('cart', [
            'title' => 'Twój koszyk - Amazonix',
            'cart' => $cartData,
            'scripts' => ['static/js/cart.js'],
            'styles' => ['public/static/css/style.css']
        ]);
    }

    public function update(): void
    {
        if (!isset($_SESSION['user_id'])) {
            header('Location: /amazonix/login');
            exit;
        }

        $itemId = (int)($_POST['item_id'] ?? 0);
        $currentQuantity = (int)($_POST['current_quantity'] ?? 1);
        $action = $_POST['action'] ?? '';

        if ($action === 'increase') {
            $newQuantity = $currentQuantity + 1;
        } elseif ($action === 'decrease') {
            $newQuantity = $currentQuantity - 1;
        } else {
            $newQuantity = (int)($_POST['quantity'] ?? 1);
        }

        if ($newQuantity <= 0) {
            $_POST['item_id'] = $itemId;
            $this->remove();
            return;
        }

        $result = $this->api->updateCart($itemId, $newQuantity);

        if ($result && isset($result['success'])) {
            $_SESSION['cart_message'] = 'Koszyk zaktualizowany';
        } else {
            $_SESSION['cart_error'] = $result['error'] ?? 'Błąd aktualizacji';
        }

        header('Location: /amazonix/cart');
        exit;
    }

    public function remove(): void
    {
        if (!isset($_SESSION['user_id'])) {
            header('Location: /amazonix/login');
            exit;
        }

        $itemId = (int)($_POST['item_id'] ?? 0);

        if ($itemId === 0) {
            $_SESSION['cart_error'] = 'Nieprawidłowy produkt';
            header('Location: /amazonix/cart');
            exit;
        }

        $result = $this->api->removeFromCart($itemId);

        if ($result && isset($result['success'])) {
            $_SESSION['cart_message'] = 'Produkt usunięty z koszyka';
        } else {
            $_SESSION['cart_error'] = $result['error'] ?? 'Błąd usuwania';
        }

        header('Location: /amazonix/cart');
        exit;
    }
}
```

h. Gałąź: feature/account-page

Celem tej gałęzi było stworzenie panelu profilu użytkownika. Skupiono się na bezpiecznym pobieraniu danych z JWT i ich prezentacji w profilu oraz umożliwieniu edycji informacji o koncie. Jest to kluczowy element domykający funkcjonalność konta użytkownika w systemie.

Najważniejsze pliki:

src/Controllers/AccountController.php

```php
<?php
    declare(strict_types=1);

    namespace App\Controllers;

    class AccountController extends Controller
    {
        public function showAccount(): void
        {
            if(!isset($_SESSION['JWT_TOKEN'])) {
                $this->redirect->to('/login');
            }

            $user = $this->jwt->decode($_SESSION['JWT_TOKEN']);



            if($user == []) {
                $_SESSION['toast_message'] = $hashedAccountData['error'] ?? 'Błąd wyswietlania danych';
                $this->redirect->to('/');
            }

            $this->view->renderPage('account/main', [
                'title' => 'Moje konto',
                'user' => $user,
                'scripts' => [
                    'static/js/account.js'
                ],
                'styles' => [
                    'public/static/css/style.css',
                    'public/static/css/account/nav.css'

                ]
            ]);
        }

    }
```

i. Gałąź: feature/o-nas

Gałąź dedykowana rozbudowie sekcji informacyjnej serwisu. Wprowadzono podstronę 'O nas' oraz rozbudowany moduł 'Kariera' z ofertami pracy i stanowiskami. Zaimplementowano integrację z Google Maps (szkielet) oraz ujednolicono system nagłówków i stopek dla nowych podstron, nadając serwisowi profesjonalny charakter.

Najważniejsze pliki:

src/Controllers/FooterController.php

```php
<?php

    namespace App\Controllers;

class FooterController extends Controller
{
    public function aboutUs(): void
    {
        try {
            $user = $this->jwt->decode($_SESSION['JWT_TOKEN'] ?? '');
        } catch (\Exception $e) {
            $user = null;
        }

        $this->view->renderPage('about-us', [
            'title' => 'O nas',
            'user' => $user,
            'scripts' => [],
            'styles' => [
                'static/css/about-us.css'
            ]
        ]);
    }
    public function careers(): void
    {
        try {
            $user = $this->jwt->decode($_SESSION['JWT_TOKEN'] ?? '');
        } catch (\Exception $e) {
            $user = null;
        }

        $offers = $this->api->request(RequestType::GET, '/careers');

        $this->view->renderPage('careers', [
            'title' => 'Kariera',
            'user' => $user,
            'offers' => $offers ?? [],
            'scripts' => [],
            'styles' => []
        ]);
    }
}
```

j. Gałąź: bugfix/strict-types

Branch techniczny, którego celem było podniesienie jakości i stabilności kodu poprzez wprowadzenie ścisłego typowania (strict types) w kluczowych plikach PHP. Zmiany te eliminują błędy rzutowania typów w czasie wykonywania i ułatwiają statyczną analizę kodu.